

Plotting of NAHES dataset

Hamed

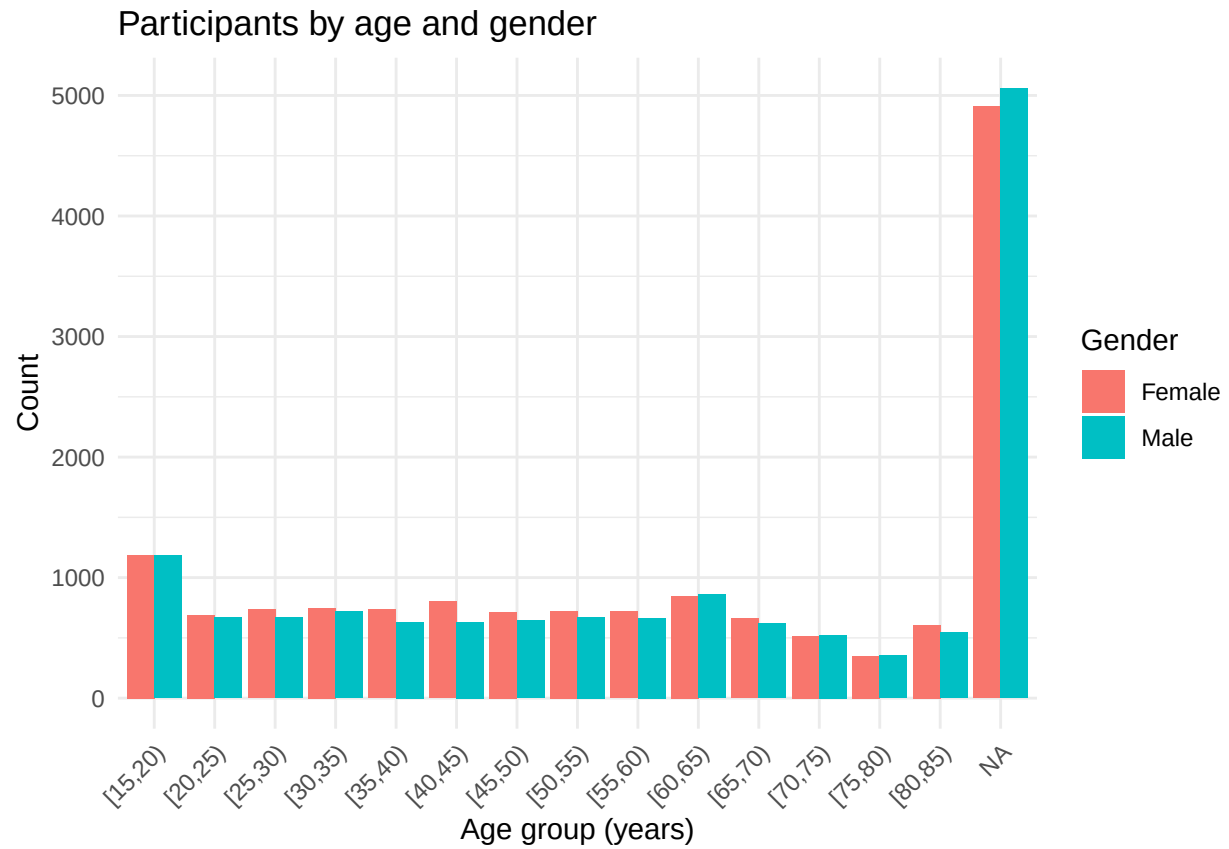
2026-09-26

Exercise 1

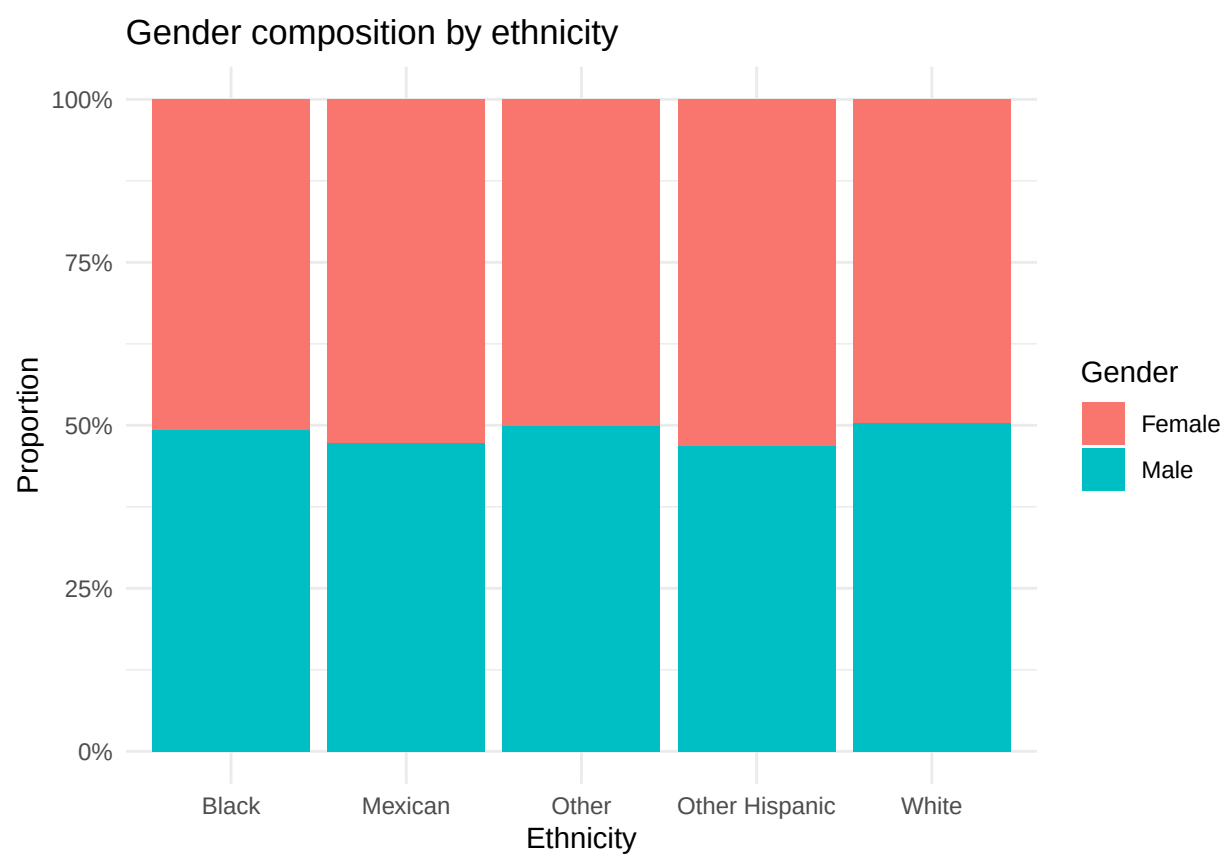
```
hanes <- hanes %>%
  mutate(
    age_group = cut(age, breaks = seq(15, 90, 5), right = FALSE),
    gender = factor(gender)
  )

p1 <- ggplot(hanes, aes(age_group, fill = gender)) +
  geom_bar(position = "dodge") +
  labs(
    title = "Participants by age and gender",
    x = "Age group (years)",
    y = "Count",
    fill = "Gender"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
p2 <- ggplot(hanes, aes(ethnicity_1, fill = gender)) +
  geom_bar(position = "fill") +
  scale_y_continuous(labels = scales::percent) +
  labs(
    title = "Gender composition by ethnicity",
    x = "Ethnicity",
    y = "Proportion",
    fill = "Gender"
  ) +
  theme_minimal()

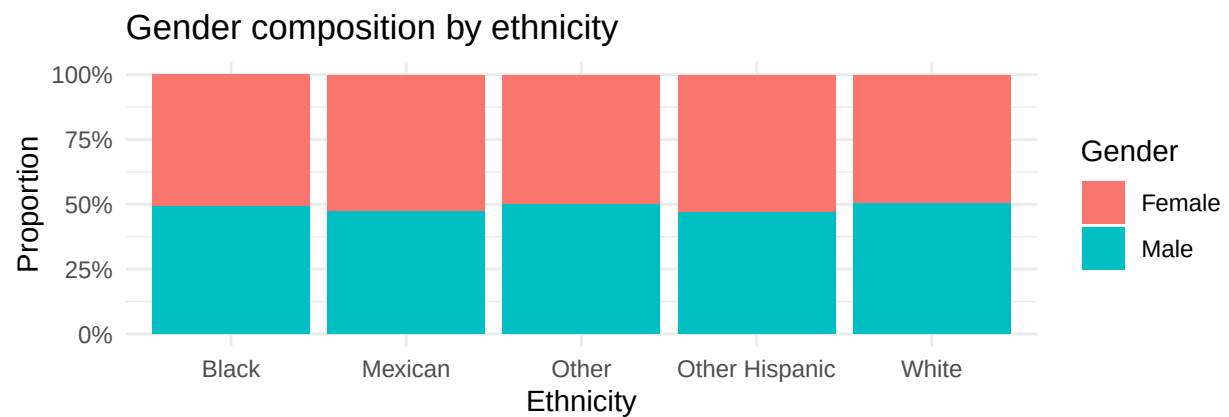
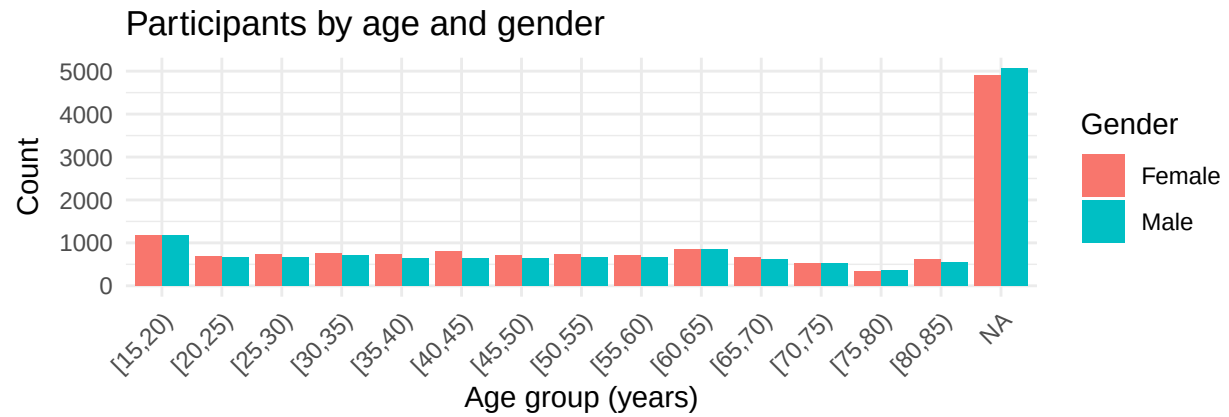
p1
```



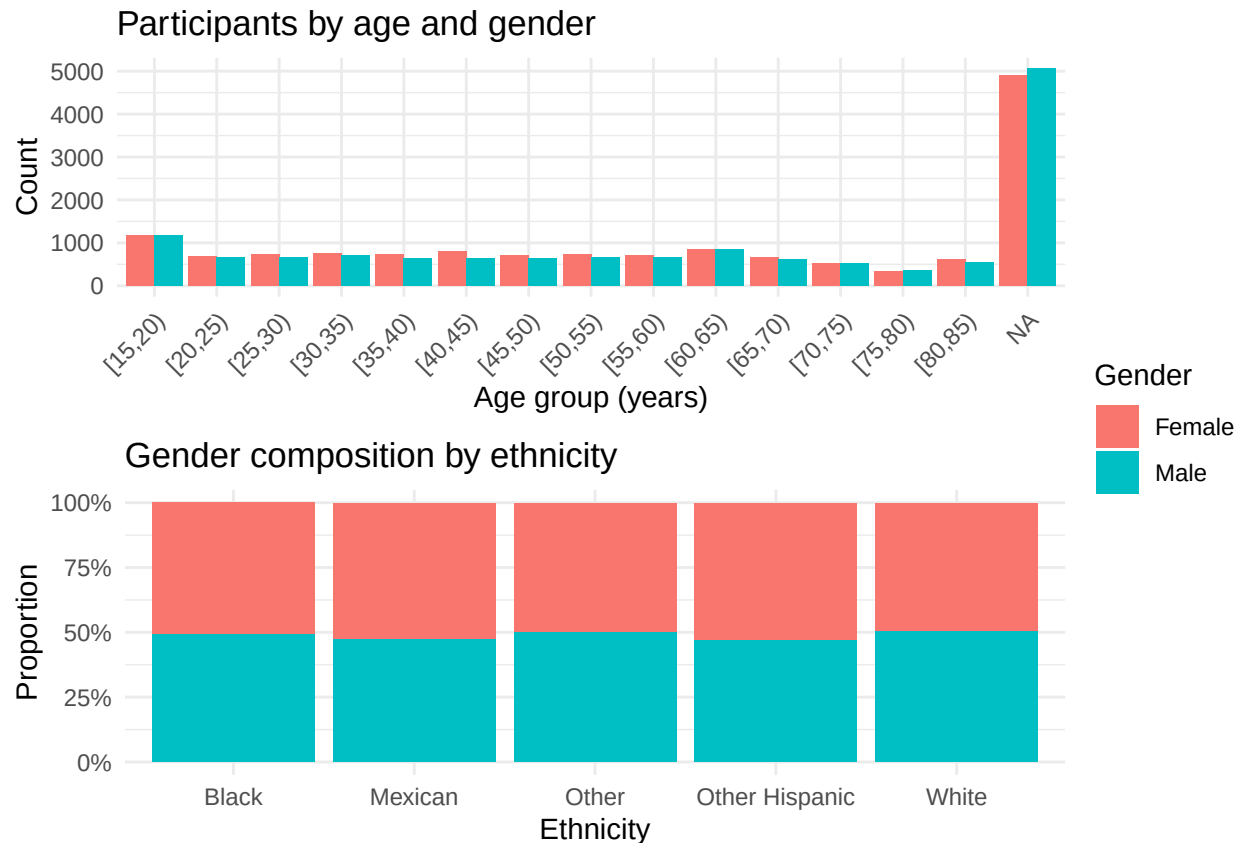
p2



```
plot_grid(p1, p2, ncol = 1, align = "v")
```



```
ggpubr::ggarrange(
  p1,
  p2,
  ncol = 1,
  common.legend = TRUE,
  legend = "right"
)
```



For these figures, I prefer `plot_grid()` because it gives both plots the same width while leaving each legend beside its own plot. The vertical arrangement also gives the long ethnicity labels enough space. `ggarrange()` produces a cleaner shared legend, but the separate legends make the two mappings easier to check while I am still developing the figures.

Exercise 2

```
missing_counts <- hanes %>%
  summarise(
    across(
      c(age, gender, ethnicity_1, ethnicity_2),
      ~ sum(is.na(.))
    )
  )
missing_counts
```

```
## # A tibble: 1 x 4
##   age gender ethnicity_1 ethnicity_2
##   <int> <int>         <int>         <int>
## 1     0     0           0           0
```

```
age_plot <- ggplot(hanes, aes(age)) +
  geom_histogram(
```

```

    binwidth = 5,
    fill = "steelblue",
    colour = "white"
  ) +
  labs(
    title = "Age distribution",
    x = "Age (years)",
    y = "Count"
  ) +
  theme_minimal()

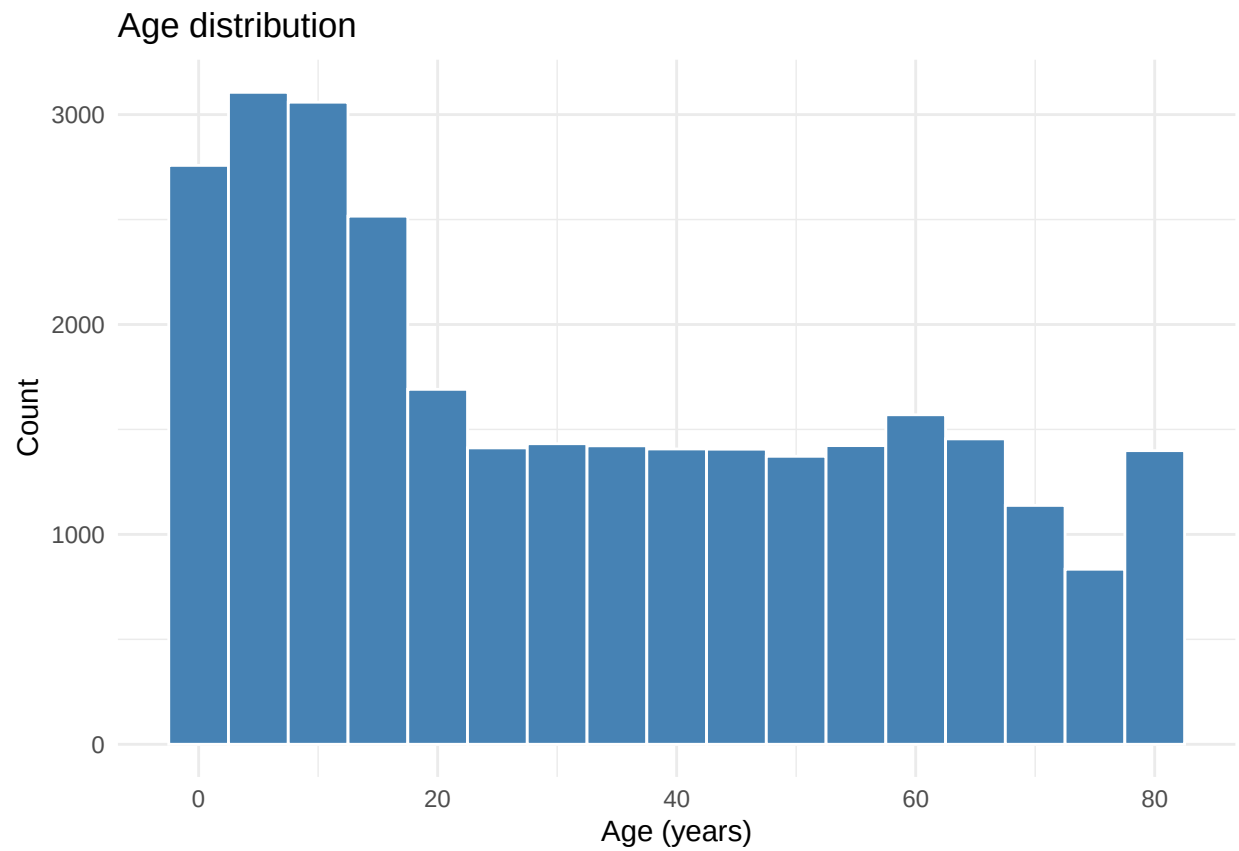
gender_plot <- ggplot(hanes, aes(gender, fill = gender)) +
  geom_bar() +
  labs(
    title = "Participants by gender",
    x = "Gender",
    y = "Count"
  ) +
  theme_minimal() +
  theme(legend.position = "none")

e1 <- ggplot(hanes, aes(ethnicity_1, fill = ethnicity_1)) +
  geom_bar() +
  labs(
    title = "Ethnicity variable 1",
    x = "Ethnicity",
    y = "Count"
  ) +
  theme_minimal() +
  theme(
    legend.position = "none",
    axis.text.x = element_text(angle = 35, hjust = 1)
  )

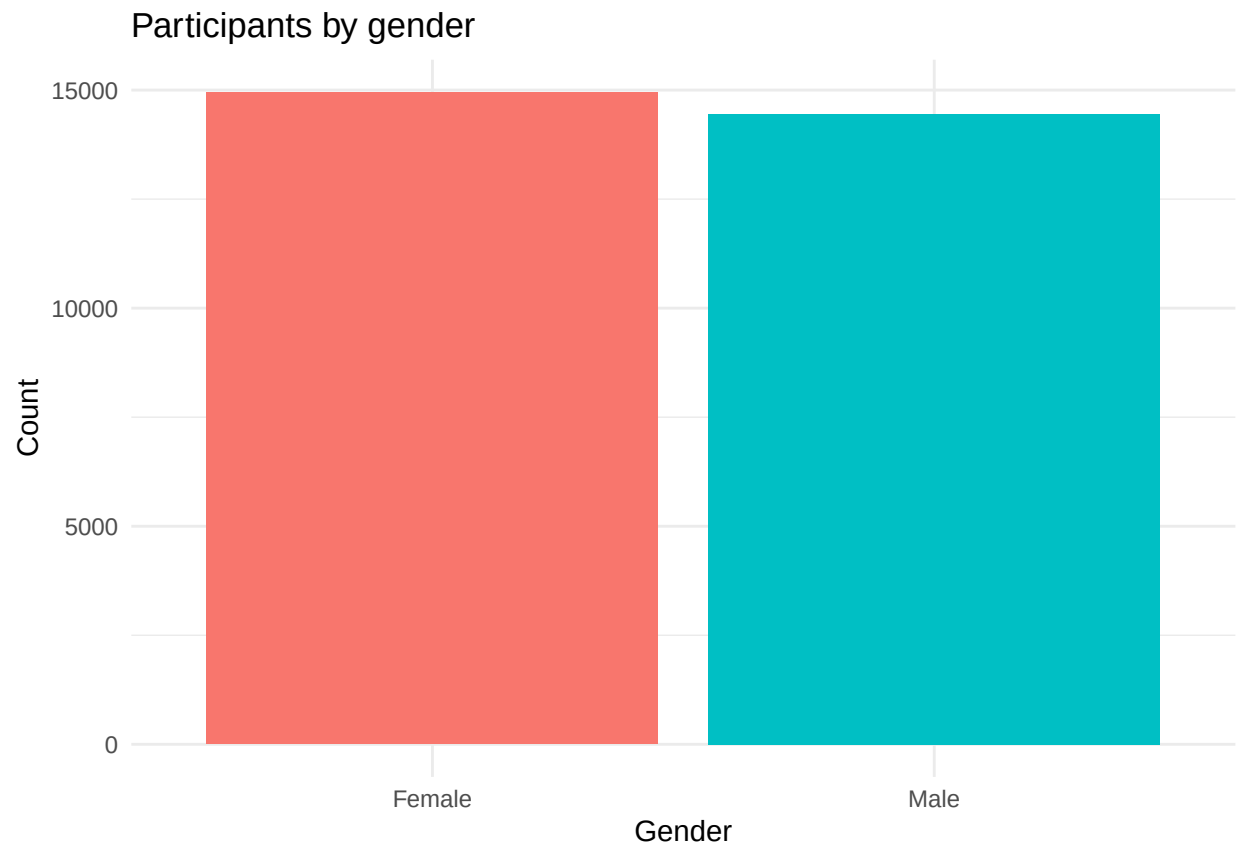
e2 <- ggplot(hanes, aes(ethnicity_2, fill = ethnicity_2)) +
  geom_bar() +
  labs(
    title = "Ethnicity variable 2",
    x = "Ethnicity",
    y = "Count"
  ) +
  theme_minimal() +
  theme(
    legend.position = "none",
    axis.text.x = element_text(angle = 35, hjust = 1)
  )

age_plot

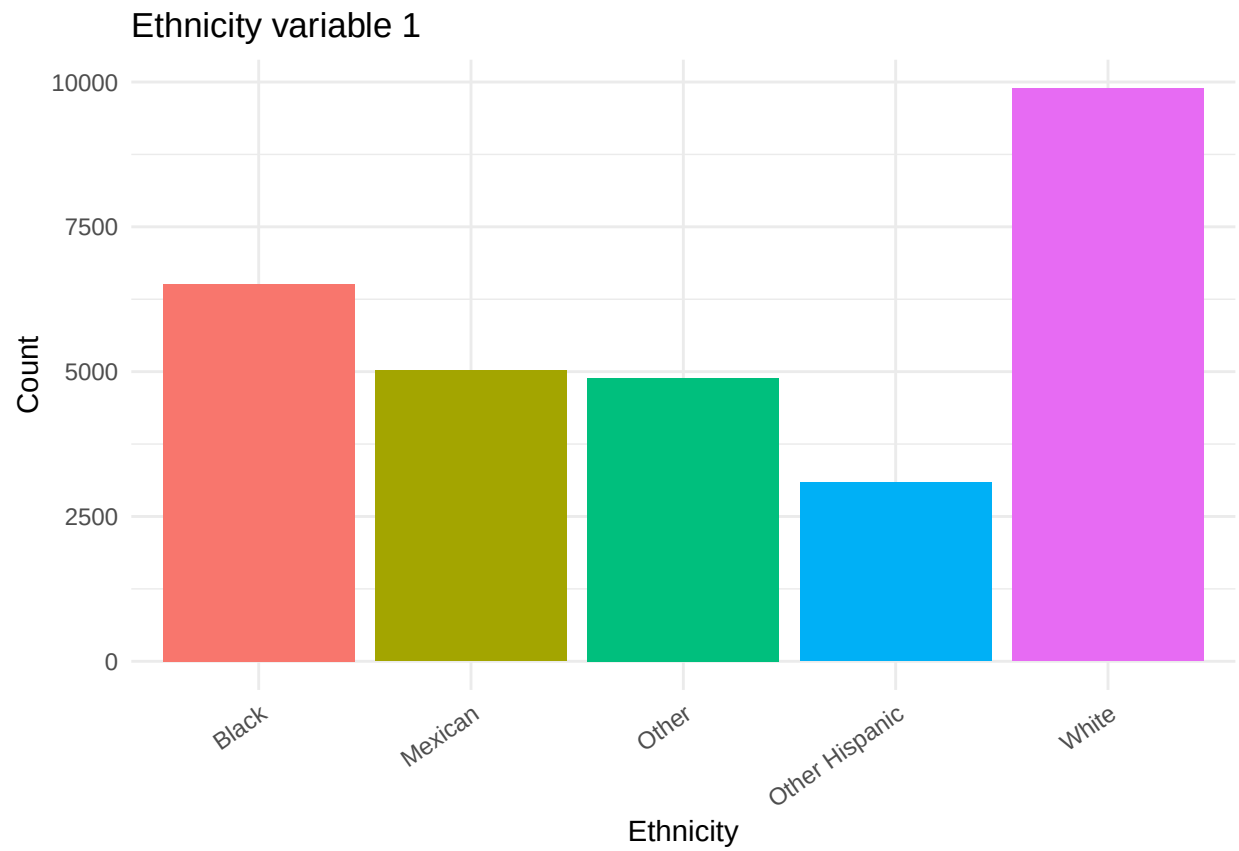
```



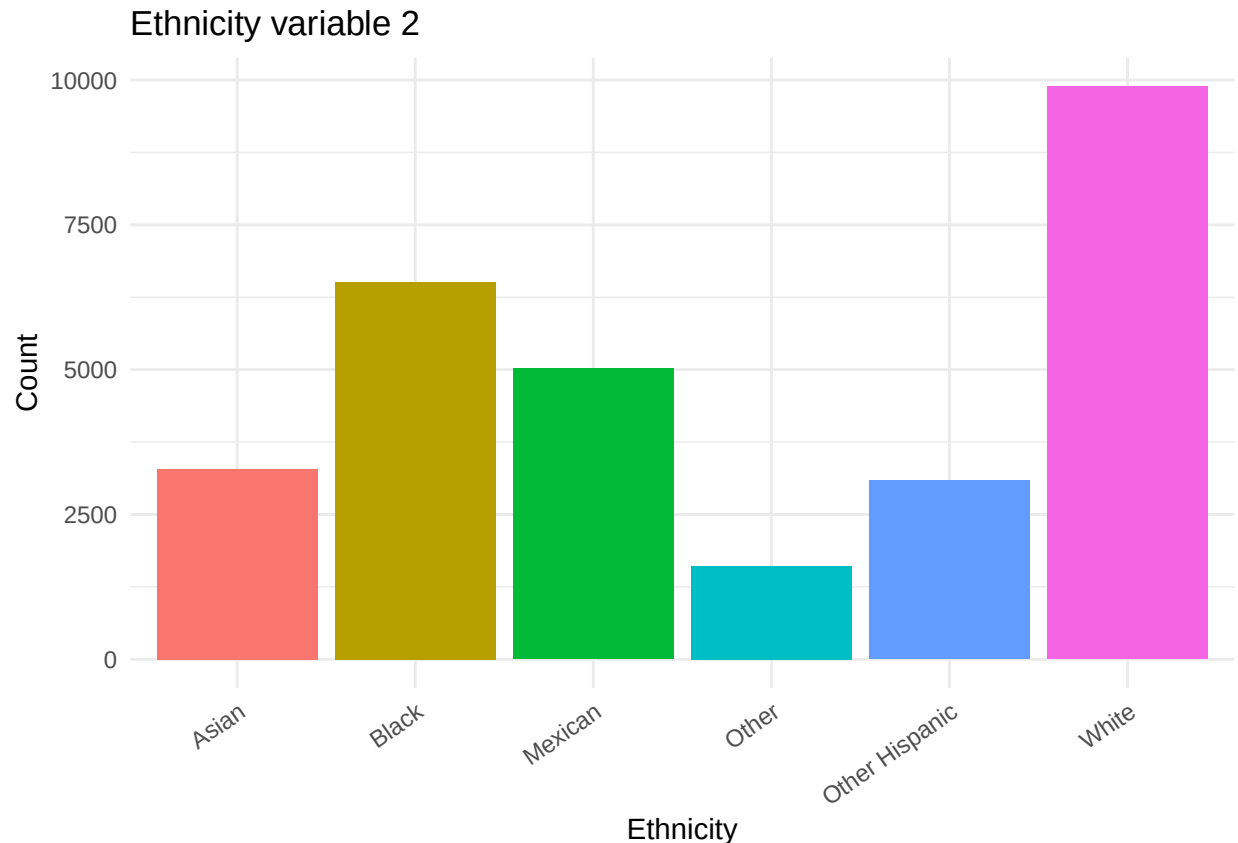
gender_plot



e1



e2



```
ggsave(here("A_age_distribution.png"), age_plot)
ggsave(here("A_gender_distribution.png"), gender_plot)
ggsave(here("A_ethnicity_1.png"), e1)
ggsave(here("A_ethnicity_2.png"), e2)
```

The age plot shows that the sample includes children, working-age adults, and older adults, rather than being concentrated in one narrow age range. Gender is summarized with counts because it is categorical. The missing-value table above reports the exact number of missing observations for all four variables.

I would retain `ethnicity_2`. It separates the Asian category from the broader “Other” group, so it preserves more information than `ethnicity_1`. I would remove `ethnicity_1` from the working dataset after documenting this choice.

Exercise 3

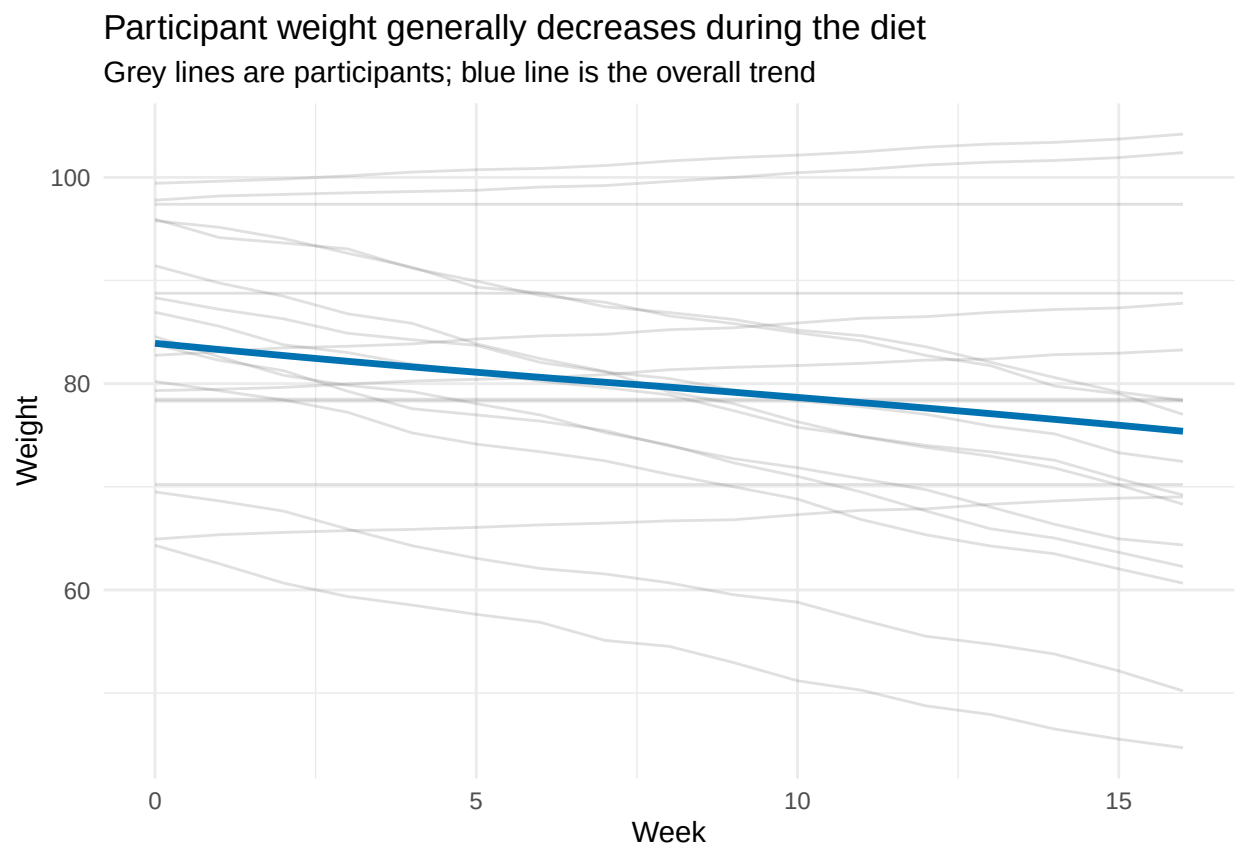
```
diet2 <- diet %>%
  group_by(Participant) %>%
  arrange(Week, .by_group = TRUE) %>%
  mutate(baseline = first(Weight)) %>%
  ungroup()

ggplot(diet2, aes(Week, Weight, group = Participant)) +
  geom_line(alpha = 0.2, colour = "grey40") +
```

```

geom_smooth(
  aes(group = 1),
  method = "loess",
  se = FALSE,
  colour = "#0072B2",
  linewidth = 1.2
) +
labs(
  title = "Participant weight generally decreases during the diet",
  subtitle = "Grey lines are participants; blue line is the overall trend",
  x = "Week",
  y = "Weight"
) +
theme_minimal()

```



The original figure assigns a different colour and legend entry to every participant, creating clutter. Transparency and one overall smooth line preserve individual trajectories while directing attention to the overall change.

Exercise 4

```

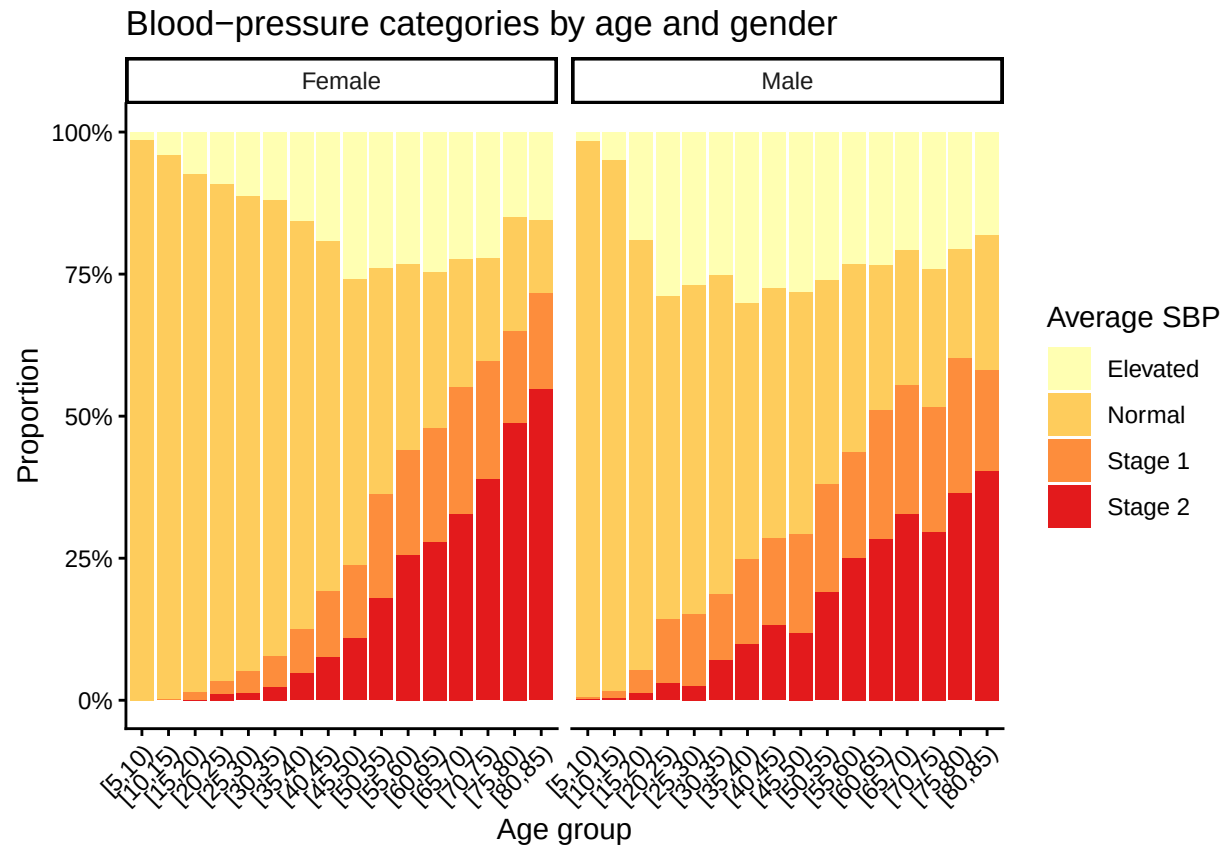
bp <- hanes %>%
  mutate(

```

```

readings_available = rowSums(
  !is.na(across(starts_with("systolic_bp_")))
),
avg_sbp = rowMeans(
  across(starts_with("systolic_bp_")),
  na.rm = TRUE
),
avg_sbp = if_else(
  readings_available == 0,
  NA_real_,
  avg_sbp
),
sbp_group = case_when(
  avg_sbp < 120 ~ "Normal",
  avg_sbp < 130 ~ "Elevated",
  avg_sbp < 140 ~ "Stage 1",
  TRUE ~ "Stage 2"
),
age_group = cut(age, breaks = seq(0, 90, by = 5), right = FALSE, include.lowest = TRUE)
) %>%
filter(!is.na(age), !is.na(gender), !is.na(avg_sbp))
ggplot(bp, aes(age_group, fill = sbp_group)) +
  geom_bar(position = "fill") +
  facet_wrap(~ gender) +
  scale_y_continuous(labels = scales::percent) +
  scale_fill_brewer(palette = "YlOrRd") +
  labs(
    title = "Blood-pressure categories by age and gender",
    x = "Age group",
    y = "Proportion",
    fill = "Average SBP"
  ) +
  theme_classic() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

```



The complete-case filter means the final distributions describe participants with age, gender, and at least one SBP reading. Ordered factors, a sequential palette, faceting, and a non-default theme make the hypertension pattern easier to compare.